Name Of Course	**Bachelor Of Computer Application (BCA)**
Semester	**4TH**
Name Of Subject	**Data Structure and Algorithm**
Subject Code	**BCA-402**
Marks	**75 Marks**
Lectures	**50 Lectures**

Unit I

1. Introductions and Overview:

1.1 Introduction
1.2 Basic technology, elementary data organization
1.3 Data structure
1.4 Data structure operation
1.5 Notation and Concept of algorithm
1.6 Complexity, time space tradeoff
1.7 Introduction to Array
1.8 Linear array
1.9 Representation of linear array in memory
1.10 Traversing linear array
1.11 Inserting and Deleting
1.12 Searching methods (Binary and linear search)

1.1 Introduction:

Introduction to Data Structures and Algorithms:

Data Structures are specialized ways to organize, manage, and store data efficiently to enable easy access and modification. Examples include arrays, linked lists, stacks, and queues. They form the backbone of how data is handled in computer science.

Algorithms are step-by-step procedures or formulas for solving problems. They are used to perform operations like searching, sorting, and optimization efficiently.

Together, Data Structures and Algorithms are essential for writing efficient and optimized programs, enabling developers to solve problems faster and with minimal resource usage. They are crucial in areas like software development, artificial intelligence, and competitive programming.

1.2 Basic technology, elementary data organization:

Here's a concise explanation:

Basic Technology and Elementary Data Organization:

Basic Technology refers to the fundamental tools and techniques used in computer systems, such as processors, memory, and storage, which support data processing and management. It also includes programming languages and software tools that enable problem-solving.

Elementary Data Organization is the way data is structured and arranged for efficient use. It involves simple structures like arrays, lists, and tables to store and access data systematically. Proper organization

helps in performing operations like searching, sorting, and updating easily.

1.3 Data structure:

Here's a concise explanation of Data Structure:

Data Structure:

A **Data Structure** is a method of organizing and storing data in a computer so that it can be used efficiently. Common types of data structures include:

Arrays:

A collection of items stored at contiguous memory locations.

Linked Lists:

A series of connected nodes where each node points to the next.

Stacks:

A structure that follows the Last In, First Out (LIFO) principle.

Queues:

A structure that follows the First In, First Out (FIFO) principle.

Data structures are essential for solving complex problems, optimizing performance, and managing large amounts of data in areas like databases, networking, and algorithms.

1.4 Data structure operation:

Here's a concise explanation of Data Structure Operations:

Data Structure Operations:

Operations on data structures are actions performed to manage and manipulate data efficiently. The key operations include:

1. Insertion:

Adding new elements to a data structure (e.g., inserting a value into an array or linked list).

2. Deletion:

Removing elements from a data structure (e.g., deleting an element from a stack or queue).

3. Traversal:

Accessing each element in the data structure to process it (e.g., printing elements of an array).

4. Searching:

Finding a specific element in the data structure (e.g., searching in a binary tree).

5. Sorting:

Rearranging elements in a specific order (e.g., ascending or descending in an array).

These operations are fundamental for problem-solving and efficient data management.

1.5 Notation and Concept of algorithm:

Here's a concise explanation for Notation and Concept of Algorithm:

Notation and Concept of Algorithm:

An **algorithm** is a step-by-step procedure to solve a specific problem in a finite amount of time. It takes an input, processes it, and produces an output.

Concept of Algorithm:

1. Finite:

Must have a definite number of steps.

2. Input/Output:

Takes input and produces an output.

3. Unambiguous:

Each step should be clear and precise.

4. Effectiveness:

Steps must be feasible to execute.

Notation of Algorithm:

Algorithms can be written using:

Pseudocode:

A language-independent representation using simple instructions.

Flowcharts:

Visual diagrams with symbols to represent steps and their flow.

Algorithms are crucial for programming, optimization, and solving problems systematically.

1.6 Complexity, time space tradeoff:

Here's a concise explanation of Complexity and Time-Space Tradeoff:

Complexity and Time-Space Tradeoff:

Complexity:

The complexity of an algorithm measures the resources it requires, mainly:

1. Time Complexity:

The amount of time an algorithm takes to execute as the input size increases. It is expressed using notations like Big-O (e.g., O(n), O(log n)).

2. Space Complexity:

The amount of memory an algorithm uses, including input data, auxiliary space, and program space.

Time-Space Tradeoff:

The **Time-Space Tradeoff** refers to the balance between time and memory usage in an algorithm. For example:

Using more memory (space) can reduce execution time, such as storing precomputed values.

Using less memory can increase execution time, like recomputing values repeatedly.

Optimizing algorithms often involves finding the best balance based on the problem's requirements.

1.7 Introduction to Array:

Here's a concise explanation of Introduction to Array:

Introduction to Array:

An **array** is a data structure used to store a collection of elements of the same data type, arranged in a contiguous block of memory. Each element in an array can be accessed using its index, which starts from 0.

Key Features of Arrays:

1. Fixed Size:

The size of an array is defined at the time of creation.

2. Efficient Access:

Elements can be quickly accessed using their index.

3. Homogeneous Data:

All elements must be of the same type (e.g., integers, strings).

Example:

If an array `A` contains `{10, 20, 30}`, `A[0]` gives `10`.

Arrays are widely used for tasks like sorting, searching, and storing sequential data in programming.

1.8 Linear array:

Here's a concise explanation of Linear Array:

Linear Array:

A **Linear Array** is a basic data structure that stores elements in a sequential and contiguous manner in memory. It is also called a **one-dimensional array** because the elements are arranged in a single row.

Key Features:

1. Fixed Size:

The number of elements is defined at the time of array creation.

2. Indexing:

Elements are accessed using an index that starts from 0.

3. Homogeneous Data:

All elements must be of the same data type.

4. Contiguous Memory:

Elements are stored in adjacent memory locations.

Example:

A linear array `A` with 5 elements: `A = [10, 20, 30, 40, 50]`

Access: `A[0]` = 10, `A[1]` = 20, etc.

Linear arrays are commonly used for simple data storage and operations like searching, sorting, and traversal.

1.9 Representation of linear array in memory:

Here's a concise explanation of the Representation of Linear Array in Memory:

Representation of Linear Array in Memory:

A **linear array** is stored in **contiguous memory locations**, meaning all elements are placed next to each other in memory. The elements can be accessed using their indices, which are mapped to specific memory addresses.

Key Points:

1. **Contiguous Memory Allocation:**

   All elements of the array are stored in consecutive memory locations. For example, if the array has 5 elements, the first element will be at the starting memory address, the second element at the next address, and so on.

2. **Indexing:**

   The index of the array determines the position of an element in memory. For an array `A` with a starting memory address `base_address`, the address of an element `A[i]` is calculated as:

   `Address of A[i] = base_address + (i * size_of_element)`

3. **Example:**

   For an array `A = [10, 20, 30, 40, 50]`, if the base address of `A[0]` is `1000` and each element takes 4 bytes, the memory addresses would be:

   `A[0]` at address `1000`
   `A[1]` at address `1004`
   `A[2]` at address `1008`, and so on.

Linear arrays allow efficient access to elements because the memory locations are predictable and calculated using the index.

1.10 Traversing linear array:

Here's a concise explanation of Traversing a Linear Array:

Traversing Linear Array:

Traversing a linear array refers to the process of visiting each element of the array exactly once, typically to perform some operation like printing or modifying the elements.

Steps for Traversing:

1. **Start from the First Element:**

   Begin at the index 0 (first element) of the array.

2. **Access Each Element:**

   Move sequentially from index 0 to the last index (`n-1`), accessing each element in turn.

3. **Perform Operations:**

   You can perform operations like printing, updating, or checking each element during traversal.

Example:

Given an array `A = [10, 20, 30, 40, 50]`, traversal involves accessing:

`A[0] = 10`
`A[1] = 20`
`A[2] = 30`
`A[3] = 40`
`A[4] = 50`

Implementation (in pseudocode):

```
for i = 0 to n-1:
    print(A[i])
```

Key Points:

Traversing is **linear** in nature, meaning it takes O(n) time, where `n` is the number of elements in the array.

It's an essential operation for tasks like searching, sorting, or processing data stored in arrays.

1.11 Inserting and Deleting:

Here's a brief explanation of Inserting and Deleting in data structures:

Inserting:

Insertion is the process of adding a new element to a data structure. The location where the new element is added depends on the type of data structure:

1. **Array:**
   Insert at a specific index, which may require shifting elements if the array is not empty.
2. **Linked List:**
   Insert at the beginning, end, or a specific position by adjusting pointers.
3. **Stack:**
   Insert at the top (push operation).
4. **Queue:**
   Insert at the rear.
5. **Binary Tree:**
   Insert according to rules, such as maintaining the binary search property.

Time Complexity:

Arrays:
O(n) (for shifting elements).

Linked Lists:
O(1) (for head/tail) or O(n) (for specific position).

Deleting:

Deletion is the process of removing an element from a data structure. The removal process also depends on the type of data structure:

1. **Array:**
   Delete at a specific index, followed by shifting elements to fill the gap.
2. **Linked List:**

Adjust pointers to skip the node being deleted.

3. Stack:

Remove from the top (pop operation).

4. Queue:

Remove from the front.

5. Binary Tree:

Delete nodes while maintaining the structure's properties.

Time Complexity:

Arrays:

O(n) (for shifting elements).

Linked Lists:

O(1) (head/tail) or O(n) (specific position).

Key Points:

Define Insertion and Deletion.

Explain with examples of arrays and linked lists.

Mention time complexities for clarity.

1.12 Searching methods (Binary and linear search):

Searching Methods (Binary and Linear Search):

1. Linear Search:

Linear search is a simple searching technique that checks each element of a data structure one by one until the desired element is found or the end is reached.

Steps:

1. Start from the first element of the array.
2. Compare each element with the target.
3. If the target is found, return its position; otherwise, continue until the last element.

Advantages:

Works on both sorted and unsorted data.

Easy to implement.

Time Complexity:

Best Case:

O(1) (if the element is at the start).

Worst Case:

O(n) (if the element is not found or at the end).

2. Binary Search:

Binary search is an efficient technique that works only on sorted data by dividing the search interval in half repeatedly.

Steps:

1. Set two pointers: low (start of array) and high (end of array).

2. Find the middle element: `mid = (low + high) // 2`.
3. Compare the middle element with the target:
    If the target equals the middle element, return its position.
    If the target is smaller, adjust the high pointer (`high = mid - 1`).
    If the target is larger, adjust the low pointer (`low = mid + 1`).
4. Repeat until the target is found or the search space is exhausted.

Advantages:

Faster than linear search for large datasets.

Time Complexity:

Best Case:

O(1) (if the element is found in the middle).

Worst Case:

O(log n) (due to halving the search space).

Comparison Table:

Feature	Linear Search	Binary Search
Data Requirement	Works on unsorted/sorted data	Requires sorted data
Time Complexity	O(n)	O(log n)
Implementation	Simple	Slightly complex
Use Case	Small datasets	Large datasets

Unit II

2. Sorting and Linked list:

2.1 Selection sort
2.2 Bubble sort
2.3 Insertion sort
2.4 Introduction to Linked list
2.5 Representation of Linked list in memory
2.6 Searching a linked list
2.7 Memory allocation, Garbage collection
2.8 Insertion and deletion in linked list

2.1 Selection sort:

Definition:

Selection Sort is a simple comparison-based sorting algorithm that repeatedly selects the smallest (or largest) element from the unsorted portion of the array and places it in its correct position.

Steps of Selection Sort:

1. **Start with the first element:** Assume it is the smallest element.
2. Compare it with all other elements in the unsorted part of the array.
3. Find the smallest element and swap it with the first element of the unsorted portion.
4. Repeat the process for the remaining unsorted part of the array until the entire array is sorted.

Example:

Sort the array `[64, 25, 12, 22, 11]` in ascending order.

Steps:

1. Initial array: `[64, 25, 12, 22, 11]`
   Find the smallest element (`11`) and swap with the first element.
   Result: `[11, 25, 12, 22, 64]`

2. Next pass: `[11, 25, 12, 22, 64]`
   Find the smallest element in the unsorted part (`12`) and swap with the second element.
   Result: `[11, 12, 25, 22, 64]`

3. Next pass: `[11, 12, 25, 22, 64]`
   Find the smallest element (`22`) and swap with the third element.
   Result: `[11, 12, 22, 25, 64]`

4. Final pass: `[11, 12, 22, 25, 64]`
   The array is now sorted.

Time Complexity:

Best Case: $O(n^2)$

Worst Case: $O(n^2)$
Space Complexity: O(1) (In-place sorting algorithm)

Advantages:
Simple to implement.
Does not require additional memory.

Disadvantages:
Inefficient for large datasets due to $O(n^2)$ complexity.

2.2 Bubble sort:

Definition:
Bubble Sort is a simple comparison-based sorting algorithm where adjacent elements are repeatedly compared and swapped if they are in the wrong order. This process continues until the entire array is sorted.

Steps of Bubble Sort:
1. Start with the first element: Compare it with the next element.
2. Swap the elements if they are in the wrong order (e.g., for ascending order, the larger element should be moved right).
3. Repeat the comparison for each adjacent pair in the array. After the first pass, the largest element "bubbles" to the end of the array.
4. Repeat the process for the remaining unsorted portion of the array.
5. Stop when no swaps are required in a pass (the array is sorted).

Example:
Sort the array `[5, 3, 8, 6, 2]` in ascending order.

Steps:
1. Initial array: `[5, 3, 8, 6, 2]`
   Compare `5` and `3` → Swap → `[3, 5, 8, 6, 2]`
   Compare `5` and `8` → No swap → `[3, 5, 8, 6, 2]`
   Compare `8` and `6` → Swap → `[3, 5, 6, 8, 2]`
   Compare `8` and `2` → Swap → `[3, 5, 6, 2, 8]`

2. Second pass: `[3, 5, 6, 2, 8]`
   Compare `3` and `5` → No swap → `[3, 5, 6, 2, 8]`
   Compare `5` and `6` → No swap → `[3, 5, 6, 2, 8]`
   Compare `6` and `2` → Swap → `[3, 5, 2, 6, 8]`

3. Third pass: `[3, 5, 2, 6, 8]`
   Compare `3` and `5` → No swap → `[3, 5, 2, 6, 8]`
   Compare `5` and `2` → Swap → `[3, 2, 5, 6, 8]`

4. Fourth pass: `[3, 2, 5, 6, 8]`
   Compare `3` and `2` → Swap → `[2, 3, 5, 6, 8]`

Final array: `[2, 3, 5, 6, 8]`

Time Complexity:

Best Case: O(n) (when the array is already sorted).

Worst Case: $O(n^2)$ (when the array is sorted in reverse order).

Space Complexity: O(1) (In-place sorting).

Advantages:

Simple to implement.

Detects sorted arrays quickly with the optimization of stopping when no swaps are made.

Disadvantages:

Inefficient for large datasets due to $O(n^2)$ complexity.

2.3 Insertion sort:

Definition:

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time by repeatedly picking an element from the unsorted portion and inserting it into its correct position in the sorted portion.

Steps of Insertion Sort:

1. Start with the second element (consider the first element as already sorted).
2. Compare the current element with the elements in the sorted portion of the array.
3. Shift all elements in the sorted portion that are greater than the current element to the right.
4. Insert the current element into its correct position.
5. Repeat the process for all remaining elements in the unsorted portion of the array.

Example:

Sort the array `[7, 4, 5, 2]` in ascending order.

Steps:

1. Initial array: `[7, 4, 5, 2]`
   First pass: Compare `4` with `7`.
   Shift `7` to the right → `[7, 7, 5, 2]`.
   Insert `4` → `[4, 7, 5, 2]`.

2. Second pass: `[4, 7, 5, 2]`
   Compare `5` with `7`.
   Shift `7` to the right → `[4, 7, 7, 2]`.
   Insert `5` → `[4, 5, 7, 2]`.

3. Third pass: `[4, 5, 7, 2]`
   Compare `2` with `7`, `5`, and `4`.
   Shift `7`, `5`, and `4` to the right → `[4, 5, 7, 7]`, `[4, 5, 5, 7]`, `[4, 4, 5, 7]`.
   Insert `2` → `[2, 4, 5, 7]`.

Final array: `[2, 4, 5, 7]`

Time Complexity:

Best Case: O(n) (when the array is already sorted).

Worst Case: $O(n^2)$ (when the array is sorted in reverse order).

Space Complexity: O(1) (In-place sorting).

Advantages:

Simple and easy to implement.

Efficient for small datasets or nearly sorted arrays.

Disadvantages:

Inefficient for large datasets due to $O(n^2)$ complexity.

.

2.4 Introduction to Linked list:

Definition:

A **Linked List** is a linear data structure where elements (called nodes) are connected using pointers. Each node consists of:

1. **Data:**
   The value stored in the node.

2. **Pointer/Link:**
   A reference to the next node in the sequence.

Unlike arrays, linked lists do not require contiguous memory allocation.

Types of Linked Lists:

1. **Singly Linked List:**
   Each node points to the next node, forming a unidirectional chain.

2. **Doubly Linked List:**
   Each node contains two pointers, one pointing to the next node and the other to the previous node.

3. **Circular Linked List:**
   The last node points back to the first node, forming a circular chain.

Features of Linked List:

1. **Dynamic Size:**
   Linked lists can grow or shrink during runtime, unlike arrays with a fixed size.

2. **Efficient Insertion/Deletion:**
   Adding or removing elements is easier and faster as no shifting is required.

3. **Sequential Access:**
   Unlike arrays, elements in a linked list must be accessed sequentially starting from the head.

Structure of a Node (Example in C):

```
struct Node {
    int data;
    struct Node* next; // Pointer to the next node
};
```

Advantages:

1. Efficient Memory Usage:

No need for pre-allocated memory like arrays.

2. Easy Insertion/Deletion:

Operations are O(1) if the pointer to the node is provided.

Disadvantages:

1. Sequential Access:

Accessing an element requires traversal, making it slower (O(n)).

2. Extra Memory:

Requires extra memory for storing pointers.

Use Cases:

Implementation of stacks, queues, and graphs.

Managing dynamic memory in operating systems.

2.5 Representation of Linked list in memory:

Definition:

In a **linked list**, each element (node) is stored in separate memory locations. These nodes are linked together using pointers, making it a non-contiguous data structure.

Node Structure:

Each node in a linked list typically consists of two parts:

1. Data Field:

Stores the actual data (e.g., an integer, string, etc.).

2. Pointer/Link Field:

Stores the address of the next node in the sequence.

Example Node Structure in C:

```
struct Node {
    int data;        // Data field
    struct Node* next; // Pointer to the next node
};
```

Memory Representation:

1. Non-Contiguous Memory:

Unlike arrays, where elements are stored in contiguous memory blocks, the nodes of a linked list are scattered across memory.

Each node is linked using a pointer, which stores the memory address of the next node.

2. Head Pointer:

The head pointer is a special pointer that stores the address of the first node in the linked list.

If the list is empty, the head pointer is set to `NULL`.

3. Last Node:

The pointer of the last node in a singly linked list points to `NULL`, indicating the end of the list.

Example:

Consider a linked list with elements: `10 → 20 → 30`.

Node 1: Data = 10, Pointer = Address of Node 2

Node 2: Data = 20, Pointer = Address of Node 3

Node 3: Data = 30, Pointer = `NULL`

Memory Representation:

Address	Data	Pointer
0X100	10	0x104
0X104	20	0x108
0X108	30	NULL

The `Head` pointer stores `0x100`, the address of the first node.

Advantages of Linked List Representation:

1. Dynamic Memory Usage:

Nodes are created only when required.

2. No Fixed Size:

Can grow or shrink as needed.

3. Efficient Insertion/Deletion:

Involves updating pointers rather than shifting elements.

Disadvantages of Linked List Representation:

1. Memory Overhead:

Extra memory is needed for storing pointers.

2. Sequential Access:

Requires traversal to access elements.

2.6 Searching a linked list:

Definition:

Searching a linked list involves traversing through its nodes to find a specific value or key. The search can either return the position of the node containing the value or confirm that the value is not present in the list.

Types of Search in Linked Lists:

1. Linear Search:

The most common method for searching a linked list.

Each node is sequentially checked starting from the head until the target value is found or the list ends.

2. Binary Search (not commonly used)**:**

Applicable only to **sorted linked lists** and usually requires conversion to an array due to lack of random access.

Steps for Linear Search:

1. Initialize a pointer (`current`) to the head of the linked list.

2. Compare the value in the `current` node with the target value.
   If they match, return the node or its position.
   Otherwise, move the pointer to the next node.

3. Repeat until the end of the list is reached (`current == NULL`).

4. If the value is not found, return an indication (e.g., -1 or "not found").

Example:

Consider the linked list: `10 → 20 → 30 → 40`.

Task: Search for the value `30`.

1. Start at the head (`10`): Compare `10` with `30` → No match.
2. Move to the next node (`20`): Compare `20` with `30` → No match.
3. Move to the next node (`30`): Compare `30` with `30` → Match found.
4. Return the position or pointer to the node.

Time Complexity:

Best Case:

O(1) (if the value is in the first node).

Worst Case:

O(n) (if the value is in the last node or not present).

Space Complexity:

O(1) (no additional memory required).

Advantages of Searching in Linked Lists:

Efficient for small or unsorted lists where insertion/deletion is frequent.

Disadvantages:

Slower compared to arrays due to the lack of random access (O(n) for all cases).

Requires traversal from the head for every search operation.

2.7 Memory allocation, Garbage collection:

Memory Allocation:

Definition:

Memory allocation refers to the process of assigning memory to a program or variable during its execution. It ensures that the program has the necessary memory space to store and manipulate data.

Types of Memory Allocation:

1. Static Memory Allocation:

Memory is allocated at compile-time.

The size of variables or arrays must be predefined.

Cannot be resized during runtime.

Example: `int arr[10];`

2. Dynamic Memory Allocation:

Memory is allocated during runtime.

The size of the memory block can be adjusted as needed.

Requires explicit allocation and deallocation by the programmer.

Example in C:

```
int* ptr = (int*)malloc(sizeof(int)); // Allocates memory dynamically
free(ptr); // Frees the memory
```

Advantages:

Dynamic memory allocation allows flexibility for data structures like linked lists, queues, and stacks.

Efficient utilization of memory for large and varying data.

Garbage Collection:

Definition:

Garbage collection is the process of automatically reclaiming memory that is no longer in use by a program. It prevents memory leaks by identifying and deallocating unused memory.

How It Works:

1. Mark-and-Sweep Algorithm:

Marks all active objects in memory.

Sweeps and removes unmarked objects (garbage).

2. Reference Counting:

Each object keeps a count of references to it.

When the count drops to zero, the memory is deallocated.

3. Generational Garbage Collection:

Objects are divided into generations (young, old).

Frequent garbage collection is performed on younger generations.

Key Features:

Automated in languages like Java and Python.

In manual systems like C/C++, the programmer must use `free()` or `delete` to deallocate memory.

Comparison:

Feature	Static Allocation	Dynamic Allocation	Garbage Collection
When	Compile-time	Runtime	Runtime
Flexibility	Fixed Size	Resizable	Automatic
Manual/Automatic	Manual	Manual	Automatic in managed languages

Advantages of Garbage Collection:

Reduces programmer errors (e.g., memory leaks, dangling pointers).

Simplifies memory management in high-level languages.

Disadvantages:

Can cause performance overhead during execution.

Requires sophisticated algorithms for efficient implementation.

2.8 Insertion and deletion in linked list:

Insertion in a Linked List:

Insertion involves adding a new node to a linked list. Depending on the position, there are three types of insertion:

1. Insertion at the Beginning:

Create a new node.

Set the new node's `next` pointer to the current `head`.

Update the `head` pointer to point to the new node.

Time Complexity: O(1).

2. Insertion at the End:

Create a new node.

Traverse the list to find the last node.

Set the last node's `next` pointer to the new node.

Set the new node's `next` pointer to `NULL`.

Time Complexity: O(n).

3. Insertion at a Specific Position:

Traverse the list to find the node just before the desired position.

Create a new node and update its `next` pointer to point to the next node.

Update the previous node's `next` pointer to point to the new node.

Time Complexity: O(n).

Example of Insertion:

Linked List: `10 → 20 → 30`

Task: Insert `15` at position 2.

1. Create a new node with data `15`.
2. Traverse to the node at position 1 (`10`).
3. Set the new node's `next` to point to the node at position 2 (`20`).
4. Update the `next` pointer of node `10` to point to the new node.

Result: `10 → 15 → 20 → 30`.

Deletion in a Linked List:

Deletion involves removing a node from the linked list. Depending on the position, there are three types of deletion:

1. Deletion from the Beginning:

Update the `head` pointer to point to the second node.

Free the memory of the first node.

Time Complexity: O(1).

2. Deletion from the End:

Traverse the list to find the second last node.

Set the second last node's `next` pointer to `NULL`.

Free the memory of the last node.

Time Complexity: O(n).

3. Deletion from a Specific Position:

Traverse the list to find the node just before the node to be deleted.

Update its `next` pointer to skip the node to be deleted.

Free the memory of the deleted node.

Time Complexity: O(n).

Example of Deletion:

Linked List: `10 → 20 → 30 → 40`

Task: Delete node with value `30`.

1. Traverse to the node with value `20`.
2. Update its `next` pointer to skip the node `30` and point to `40`.
3. Free the memory of node `30`.

Result: `10 → 20 → 40`.

Unit III

3. Stacks, Queues, Recursion:

3.1 Introduction
3.2 Stacks
3.3 Array representation of stacks
3.4 Arithmetic expression
3.5 Recursion
3.6 Queues :Memory Representation, Insertion, Deletion
3.7 Deques
3.8 Priority queue

3.1 Introduction:

1. Stack:

Definition:

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. The last element added to the stack is the first one to be removed.

Operations:

Push: Add an element to the top of the stack.
Pop: Remove the top element from the stack.
Peek/Top: Retrieve the top element without removing it.

Example:

Stack operations on an empty stack:

1. Push(10): Stack becomes `[10]`
2. Push(20): Stack becomes `[10, 20]`
3. Pop(): Stack becomes `[10]` (removes 20)

Applications:

Function call management (call stack).
Undo operations in text editors.
Expression evaluation and syntax parsing.

2. Queue:

Definition:

A **queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle. The first element added to the queue is the first one to be removed.

Operations:

Enqueue: Add an element to the rear of the queue.
Dequeue: Remove an element from the front of the queue.
Peek/Front: Retrieve the element at the front without removing it.

Types of Queues:

Simple Queue: Standard FIFO behavior.

Circular Queue: Connects the rear and front to make the structure circular.

Priority Queue: Elements are dequeued based on priority rather than order.

Example:

Queue operations on an empty queue:

1. Enqueue(10): Queue becomes `[10]`
2. Enqueue(20): Queue becomes `[10, 20]`
3. Dequeue(): Queue becomes `[20]` (removes 10)

Applications:

Task scheduling.

Breadth-first search in graphs.

Managing requests in operating systems (e.g., print queues).

3. Recursion:

Definition:

Recursion is a programming technique where a function calls itself to solve smaller subproblems of the original problem.

Structure of Recursion:

1. Base Case: The stopping condition for recursion.

2. Recursive Case: The part where the function calls itself with a smaller problem.

Example: Factorial of a number `n` (n!)

```
int factorial(int n) {
    if (n == 0) // Base case
    return 1;
    return n * factorial(n - 1); // Recursive case
}
```

For `factorial(4)`:

`4 * factorial(3)` → `3 * factorial(2)` → `2 * factorial(1)` → `1 * factorial(0) = 1`.

Applications:

Solving mathematical problems (factorials, Fibonacci series).

Tree and graph traversals (DFS).

Divide-and-conquer algorithms (e.g., merge sort, quicksort).

Comparison of Stacks, Queues, and Recursion:

Feature	Stack	Queue	Recursion
Principle	LIFO	FIFO	Function calls itself
Use Case	Undo, expressions	Scheduling, BFS	Divide and conquer
Data Structure	Abstract	Abstract	Functional concept

3.2 Stacks:

Definition:

A **stack** is a linear data structure that operates on the **LIFO (Last In, First Out)** principle. This means the last element added to the stack is the first one to be removed. It is used to store and manage data in a specific order, making it ideal for tasks like reversing data, backtracking, and managing function calls.

Basic Operations:

1. **Push:** Adds an element to the top of the stack.
2. **Pop:** Removes the top element from the stack.
3. **Peek/Top:** Retrieves the top element without removing it.
4. **IsEmpty:** Checks if the stack is empty.
5. **IsFull** (optional)**:** Checks if the stack is full (used in fixed-size stacks).

Stack Representation:

A stack can be implemented using:

1. **Arrays:** Fixed-size stack with a predefined size.
2. **Linked Lists:** Dynamic stack that grows or shrinks as needed.

Example (Array-Based Stack in C):

```
#define MAX 100
int stack[MAX];
int top = -1;

void push(int value) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = value;
}

int pop() {
    if (top == -1)
        printf("Stack Underflow\n");
    else
        return stack[top--];
}
```

Examples of Stack Operations:

1. **Push:** Adding elements to a stack.
   Stack before: `[]`
   Push(10): `[10]`
   Push(20): `[10, 20]`

2. **Pop:** Removing elements from a stack.
   Stack before: `[10, 20]`
   Pop(): `[10]` (removes 20).

3. **Peek:** Retrieving the top element without removing it.
   Stack: `[10, 20]`
   Peek(): `20`.

Applications of Stacks:

1. **Function Call Management:**
   Used in the call stack for managing function calls and returns.

2. **Undo Operations:**
   For backtracking in text editors or applications.

3. **Expression Evaluation:**
   Used in converting and evaluating expressions (infix, postfix).

4. **Parsing:**
   Helps in syntax parsing for programming languages.

5. **Depth-First Search (DFS):**
   Used in graph traversal algorithms.

Advantages of Stacks:

Simple and efficient for managing data with LIFO access.
Helps in implementing recursive algorithms.

Disadvantages of Stacks:

Limited access: Only the top element can be accessed.
Fixed size (for array-based stacks) may cause overflow.

3.3 Array representation of stacks:

Definition:

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. It can be implemented using an array where elements are stored in a contiguous block of memory. The top of the stack is managed using an index, which tracks the last element added.

Structure of Array-Based Stack:

1. **Array:** Used to store the elements of the stack.
2. **Top:** A variable that keeps track of the index of the topmost element in the stack.
   Initially set to `-1` to indicate an empty stack.
3. **Capacity:** The maximum size of the array (fixed).

Operations in Array Representation:

1. **Push (Insert an element):**
   Check if the stack is full (top == capacity - 1). If true, output "Stack Overflow."
   Otherwise, increment the `top` by 1 and insert the new element at `stack[top]`.
   Time Complexity: O(1).

2. **Pop (Remove an element):**
   Check if the stack is empty (top == -1). If true, output "Stack Underflow."
   Otherwise, remove the element at `stack[top]` and decrement `top` by 1.
   Time Complexity: O(1).

3. **Peek (Retrieve the top element):**
   Return the element at `stack[top]` without removing it.

4. **IsEmpty:**
   Return true if `top == -1`.

5. **IsFull:**
   Return true if `top == capacity - 1`.

Example of Array Representation:

Consider a stack with a capacity of 5.

Initial State:

Array: `[_, _, _, _, _]`
`top = -1` (empty stack).

Push Operations:

1. **Push(10):**
   `top = 0`, Array: `[10, _, _, _, _]`

2. **Push(20):**
   `top = 1`, Array: `[10, 20, _, _, _]`

Pop Operation:

1. **Pop():**
   Removes `20`.
   `top = 0`, Array: `[10, _, _, _, _]`.

Implementation (C Code):

```
#include <stdio.h>
#define MAX 100
        int stack[MAX];
        int top = -1;

        void push(int value) {
                if (top == MAX - 1)
                        printf("Stack Overflow\n");
                else
                        stack[++top] = value;
        }

        int pop() {
                if (top == -1)
                        printf("Stack Underflow\n");
                else
                        return stack[top--];
        }

        int peek() {
                if (top == -1)
                        printf("Stack is Empty\n");
                else
                        return stack[top];
        }
```

Advantages of Array-Based Stacks:

1. Simple Implementation:

Easy to implement and understand.

2. Access Speed:

Constant time for push and pop operations (O(1)).

Disadvantages:

1. Fixed Size:

The size of the stack is fixed, which may lead to **stack overflow** if the array is full or **wasted space** if the array is underutilized.

2. Static Memory Allocation:

Cannot resize during runtime.

3.4 Arithmetic expression:

Definition:

An **arithmetic expression** is a combination of **operands** (numbers, variables) and **operators** (+, -, *, /, etc.) arranged in a specific order to calculate a value. These expressions are widely used in programming and

mathematics to perform computations.

Components of an Arithmetic Expression:

1. Operands:

The values on which operations are performed.

Examples: Numbers (e.g., 5, 10), variables (e.g., `x`, `y`).

2. Operators:

Symbols that specify the type of operation to be performed.

Examples:

Arithmetic: `+`, `-`, `*`, `/`, `%`

Relational: `<`, `>`, `==`

Logical: `&&`, `||`

3. Parentheses:

Used to group parts of the expression and control the order of operations.

Example: `(a + b) * c`.

Types of Arithmetic Expressions:

1. Infix Expression:

The operator is placed between the operands.

Example: `a + b`.

Commonly used by humans but requires more processing for evaluation by computers.

2. Prefix Expression (Polish Notation):

The operator precedes the operands.

Example: `+ a b`.

Advantage: No need for parentheses to enforce operator precedence.

3. Postfix Expression (Reverse Polish Notation):

The operator is placed after the operands.

Example: `a b +`.

Easy for computers to evaluate using stacks.

Evaluation of Arithmetic Expressions:

Rules for Evaluation:

1. Operator Precedence:

Determines the order in which operations are performed.

Highest precedence: `*`, `/`, `%`

Lowest precedence: `+`, `-`

2. Associativity:

Determines the direction of evaluation when operators have the same precedence.

Left-to-right for most operators (e.g., `*`, `+`).

3. Parentheses:

Override default precedence.

Example of Expression Evaluation:

Expression: `3 + 5 * 2 - 8 / 4`

1. Perform multiplication and division first:

   `5 * 2 = 10`

   `8 / 4 = 2`

2. Perform addition and subtraction from left to right:

   `3 + 10 = 13`

   `13 - 2 = 11`

Result: `11`.

Applications of Arithmetic Expressions:

1. Programming:

Used in calculations, decision-making, and loops.

2. Compilers:

Arithmetic expressions are parsed and converted into machine code for execution.

3. Mathematics:

Solving equations, financial calculations, and engineering problems.

3.5 Recursion:

Definition:

Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem. It continues until a specified condition (called the base case) is met, which stops the recursion.

Structure of Recursion:

1. Base Case:

The stopping condition that prevents infinite recursion.

2. Recursive Case:

The part of the function where it calls itself with a smaller problem.

Example of Recursion:

Factorial of a Number (n!):

Factorial is defined as:

$$\boldsymbol{n}! = \boldsymbol{n} \times (\boldsymbol{n} - \mathbf{1})!$$

Base case: $\mathbf{0}! = \mathbf{1}$

Recursive Function (C Code):

```
int factorial(int n) {
    if (n == 0) // Base case
        return 1;
    else
        return n * factorial(n - 1); // Recursive case
}
```

Example Calculation for $factorial(4)$**:**

1. $\boldsymbol{factorial(4) = 4 \times factorial(3)}$

2. $\boldsymbol{factorial(3) = 3 \times factorial(2)}$

3. $\boldsymbol{factorial(2) = 2 \times factorial(1)}$

4. $\boldsymbol{factorial(1) = 1 \times factorial(0)}$

5. $\boldsymbol{factorial(0) = 1\ (base\ class)}$

Result: $4! = 4 \times 3 \times 2 \times 1 = 24$

Types of Recursion:

1. Direct Recursion:

A function calls itself directly.
Example: `factorial(n)` above.

2. Indirect Recursion:

A function calls another function, which in turn calls the first function.
Example: Function `A` calls `B`, and `B` calls `A`.

Advantages of Recursion:

1. Simplifies code for problems like tree traversal, Fibonacci series, and divide-and-conquer algorithms.
2. Reduces complex iterative logic to simple function calls.

Disadvantages of Recursion:

1. Memory-Intensive:

Each recursive call consumes stack memory, which may lead to a **stack overflow** for deep recursions.

2. Slower Execution:

Recursive functions involve overhead due to repeated function calls.

Applications of Recursion:

1. Mathematics:

Solving factorial, Fibonacci, power, and combination problems.

2. Tree Traversals:

Used in Depth-First Search (DFS) for binary trees and graphs.

3. Divide and Conquer Algorithms:

Examples include quicksort and mergesort.

4. Dynamic Programming:

Problems like the knapsack problem or memoization can use recursion.

3.6 Queues :Memory Representation, Insertion, Deletion:

Definition of Queue:

A **queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle. This means the first element added to the queue is the first one to be removed.

Memory Representation of Queues:

1. Array-Based Representation:

The queue is implemented as a fixed-size array.

Two pointers are used:

Front: Points to the first element of the queue.

Rear: Points to the last element of the queue.

Initial state:

`Front = -1` (empty queue), `Rear = -1`.

Example (Array of size 5):

Queue: [_, _, _, _, _]

Front: -1, Rear: -1 (Empty)

2. Linked List Representation:

A queue is implemented as a dynamic structure using nodes.

Each node contains:

Data: Value of the element.

Pointer: Points to the next node.

Two pointers track the **front** and **rear** of the queue.

Example:

Front --> [10|*] --> [20|*] --> [30|NULL]

Rear: Points to the last node (30).

Insertion in a Queue (Enqueue):

Steps for Array Representation:

1. Check Overflow:

If `Rear == MaxSize - 1`, output "Queue Overflow."

2. If the queue is empty (`Front == -1`), set both `Front` and `Rear` to 0.

3. Insert the new element at `Queue[Rear]` and increment `Rear` by 1.

Steps for Linked List Representation:

1. Create a new node and assign the data.

2. If the queue is empty (`Front == NULL`), set both `Front` and `Rear` to the new node.

3. Otherwise, set `Rear->next` to the new node and update `Rear` to the new node.

Example (Array)**:**

1. Queue before: `[_, _, _, _, _]`, `Front = -1`, `Rear = -1`.
2. Enqueue(10): `[10, _, _, _, _]`, `Front = 0`, `Rear = 0`.
3. Enqueue(20): `[10, 20, _, _, _]`, `Front = 0`, `Rear = 1`.

Deletion in a Queue (Dequeue):

Steps for Array Representation:

1. Check Underflow:

If `Front == -1`, output "Queue Underflow."

2. Retrieve the element at `Queue[Front]`.
3. If the queue has only one element (`Front == Rear`), reset `Front = Rear = -1` (empty queue).
4. Otherwise, increment `Front` by 1 to remove the element logically.

Steps for Linked List Representation:

1. If the queue is empty (`Front == NULL`), output "Queue Underflow."
2. Retrieve the element at `Front`.
3. Move `Front` to the next node. If `Front == NULL`, set `Rear = NULL` (empty queue).
4. Free the memory of the removed node.

Example (Array)**:**

1. Queue before: `[10, 20, _, _, _]`, `Front = 0`, `Rear = 1`.
2. Dequeue(): Removes `10`. Queue becomes `[_, 20, _, _, _]`, `Front = 1`, `Rear = 1`.

Advantages and Disadvantages:

Advantages:

1. Efficient Access:

FIFO access is useful for scheduling and task management.

2. Dynamic Size:

Linked list representation can grow or shrink as needed.

Disadvantages:

1. Array Overflow:

Fixed-size queues may lead to overflow even with empty spaces (solved using circular queues).

2. Memory Overhead:

Linked list representation uses extra memory for pointers.

3.7 Deques:

Definition:

A **deque** (pronounced "deck") or **double-ended queue** is a linear data structure that allows insertion and deletion of elements from **both ends**: the front and the rear. Unlike standard queues (FIFO), deques provide

greater flexibility.

Types of Deques:

1. Input-Restricted Deque:

Insertion is allowed at only one end (rear).

Deletion can occur from both ends.

2. Output-Restricted Deque:

Deletion is allowed at only one end (front).

Insertion can occur at both ends.

Memory Representation:

Deques can be implemented using:

1. Arrays:

Fixed-size implementation with circular indexing to efficiently manage space.

2. Linked Lists:

Dynamic implementation with nodes having pointers to both the previous and next elements.

Array Representation:

Front: Points to the index of the front element.

Rear: Points to the index of the rear element.

Circular indexing ensures efficient use of memory.

Example:

Deque: [_, 10, 20, _, _]

Front: 1, Rear: 2

Linked List Representation:

Each node contains:

Data: The value of the element.

Previous and Next Pointers: To point to adjacent nodes.

Example:

Front --> [10] <--> [20] <--> [30] <-- Rear

Basic Operations in a Deque:

1. Insert at Front:

For arrays, decrement the `front` index (circularly) and place the element.

For linked lists, create a new node and update pointers.

2. Insert at Rear:

For arrays, increment the `rear` index (circularly) and place the element.

For linked lists, create a new node and update pointers.

3. Delete from Front:

For arrays, increment the `front` index (circularly).

For linked lists, move the `front` pointer to the next node and free the previous one.

4. Delete from Rear:

For arrays, decrement the `rear` index (circularly).

For linked lists, move the `rear` pointer to the previous node and free the last node.

Example Operations (Array):

Initial State:

Deque: `[_, _, _, _, _]`

Front = -1, Rear = -1

Insertion:

1. Insert 10 at rear: `[10, _, _, _, _]`, Front = 0, Rear = 0
2. Insert 20 at rear: `[10, 20, _, _, _]`, Front = 0, Rear = 1
3. Insert 5 at front: `[_, 5, 10, 20, _]`, Front = 3, Rear = 1

Deletion:

1. Delete from front: `[_, _, 10, 20, _]`, Front = 2, Rear = 1
2. Delete from rear: `[_, _, 10, _, _]`, Front = 2, Rear = 0

Applications of Deques:

1. Scheduling and Caching:

Deques are used in algorithms like the Least Recently Used (LRU) cache.

2. Sliding Window Problems:

Efficiently manage sliding window calculations in arrays.

3. Expression Evaluation:

Used in evaluating complex expressions and implementing undo/redo operations.

4. Priority Handling:

Can act as both stack and queue depending on use cases.

Advantages:

1. Flexibility:

Allows insertion and deletion from both ends.

2. Efficient Memory Usage:

With circular arrays or linked lists.

Disadvantages:

1. Complex Implementation:

Requires careful pointer or index management.

2. Fixed Size:

Array-based deques may have overflow issues if the size is not dynamic.

3.8 Priority queue:

Definition:

A **priority queue** is an abstract data structure similar to a queue, but with an added feature: **each element is associated with a priority**. In a priority queue, elements with higher priority are dequeued before elements with lower priority, regardless of the order in which they were added.

Characteristics of a Priority Queue:

1. Priority-Based Processing:

The element with the highest priority is removed first.

If two elements have the same priority, the one enqueued first is dequeued first (this depends on implementation).

2. Not Always FIFO:

Unlike a normal queue, the order of removal is based on priority, not arrival time.

Types of Priority Queues:

1. Max-Priority Queue:

The element with the highest priority is dequeued first.

2. Min-Priority Queue:

The element with the lowest priority is dequeued first.

Memory Representation:

Priority queues can be implemented using:

1. Arrays:

Elements are stored along with their priorities.

Can be sorted (efficient dequeue) or unsorted (efficient enqueue).

2. Linked Lists:

Nodes store the element and its priority.

Can be maintained in sorted order by priority for efficient dequeue.

3. Heaps:

A binary heap (max-heap or min-heap) is the most efficient implementation for priority queues.

Operations like insertion and deletion have $O(log\ n)$ time complexity.

Basic Operations:

1. Insertion (Enqueue):

Add an element to the queue with a given priority.

In an array or linked list, this may involve placing the element in the correct position based on priority.

In a heap, the element is added to maintain the heap property.

2. Deletion (Dequeue):

Remove the element with the highest (or lowest) priority.

In a heap, the root of the heap is removed, and the heap is restructured.

Example:

Elements with Priorities:

Element	Priority
A	2
B	5
C	3

Using a Max-Priority Queue:

1. Insert elements: Queue = $B(5), C(3), A(2)$.
2. Dequeue: Remove **B** (highest priority).

   Queue after removal: $C(3), A(2)$.

Applications of Priority Queues:

1. Scheduling:

Used in CPU scheduling algorithms (e.g., shortest job first, priority scheduling).

2. Pathfinding Algorithms:

Used in algorithms like Dijkstra's and A* for selecting the next shortest or most optimal path.

3. Data Compression:

Huffman encoding uses priority queues for building optimal prefix codes.

4. Real-Time Systems:

Ensures high-priority tasks are handled first.

Advantages:

1. Efficient handling of prioritized tasks.
2. Supports dynamic changes in priority.

Disadvantages:

1. Complex implementation, especially with heaps.
2. Requires extra memory to store priorities.

Unit IV

4. Tree:

- **4.1 Introduction**
- **4.2 Terminology of Binary tree**
- **4.3 Types of Binary tree**
- **4.4 Traversing of binary tree**
- **4.5 Header Nodes, Threads**
- **4.6 General Tree Introduction**

4.1 Introduction to Trees:

Definition:

A **tree** is a hierarchical data structure consisting of **nodes** connected by **edges**. It is a non-linear structure where one node is designated as the root, and every other node is connected directly or indirectly to the root.

Key Terminology:

1. Node:

A basic unit of a tree containing data.

2. Root:

The topmost node in a tree, with no parent.

3. Edge:

A connection between two nodes.

4. Parent:

A node that has child nodes.

5. Child:

A node that is a descendant of another node.

6. Leaf Node:

A node with no children.

7. Subtree:

A portion of a tree that itself is a tree.

8. Height:

The number of edges on the longest path from the root to a leaf.

9. Depth:

The number of edges from the root to a given node.

10. Degree:

The number of children a node has.

Properties of a Tree:

1. A tree with n nodes has $n - 1$ edges.
2. There is exactly one path between any two nodes.
3. Trees are connected and acyclic (no cycles).

Types of Trees:

1. Binary Tree:

Each node has at most two children (left and right).

2. Binary Search Tree (BST):

A binary tree where the left child contains values smaller than the parent, and the right child contains values greater than the parent.

3. Full Binary Tree:

Every node has 0 or 2 children.

4. Complete Binary Tree:

All levels are completely filled except possibly the last level, which is filled from left to right.

5. Balanced Tree:

The height difference between the left and right subtrees of any node is minimal (e.g., AVL tree).

Representation of Trees in Memory:

1. Linked Representation:

Each node contains:

Data.
Pointer to the left child.
Pointer to the right child.

Example:

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

2. Array Representation:

Used for complete binary trees.

For a node at index i:

Left child is at $2i + 1$.
Right child is at $2i + 2$.
Parent is at $\lfloor (i - 1)/2 \rfloor$.

Applications of Trees:

1. Hierarchical Data Storage:

File systems, XML/HTML data.

2. Searching and Sorting:

Binary search trees, heaps.

3. Routing Algorithms:

Spanning trees in networks.

4. Expression Evaluation:

Expression trees for arithmetic operations.

Advantages of Trees:

1. Hierarchical structure simplifies complex relationships.
2. Efficient for searching, insertion, and deletion in binary search trees.

Disadvantages of Trees:

1. Complex implementation compared to linear structures.
2. Balancing can be computationally expensive for some tree types.

4.2 Terminology of Binary tree:

Definition:

A **binary tree** is a type of tree data structure in which each node has at most **two children**, referred to as the **left child** and **right child**.

Key Terminology of Binary Tree:

1. Node:

A fundamental unit of the tree that contains data and links to its children.

2. Root:

The topmost node in the binary tree.

It has no parent.

Example:

```
10
```

3. Edge:

A connection between a parent node and a child node.

Represents the relationship between nodes.

4. Parent:

A node that has one or more children.

Example:

```
10
/ \
5   20
```

Here, **10** is the parent of **5** and **20**.

5. Child:

A node that is a descendant of another node.

Example:

In the above tree, **5** and **20** are children of **10**.

6. Leaf Node:

A node that has no children (left and right pointers are NULL).

Example:

In the tree above, **5** and **20** are leaf nodes.

7. Internal Node:

A node that is not a leaf (has at least one child).

Example:

In the above tree, **10** is an internal node.

8. Level:

The depth of a node from the root, starting at level 0 for the root.

Example:

```
10  --> Level 0
/  \
5    20  --> Level 1
```

9. Height:

The number of edges on the longest path from a node to a leaf.

Height of a tree = Height of the root node.

10. Depth:

The number of edges from the root to a specific node.

11. Subtree:

A tree consisting of a node and its descendants.

Example:

In the above tree, the subtree rooted at **5** is:

```
5
```

12. Degree:

The number of children a node has.

In a binary tree, the degree of a node can be 0, 1, or 2.

13. Sibling:

Nodes that share the same parent.

Example:

In the above tree, **5** and **20** are siblings.

14. Binary Tree Types:

Full Binary Tree:

Every node has 0 or 2 children.

Complete Binary Tree:

All levels are fully filled except possibly the last, filled from left to right.

Perfect Binary Tree:

All internal nodes have 2 children, and all leaf nodes are at the same level.

Skewed Binary Tree:

All nodes have only one child (left-skewed or right-skewed).

Applications of Binary Trees:

1. Represent hierarchical data like file systems.
2. Efficient searching and sorting (binary search trees).
3. Expression evaluation in expression trees.

4.3 Types of Binary tree:

Definition:

A **binary tree** is a hierarchical data structure where each node has at most two children, referred to as the **left child** and **right child**. There are several specialized types of binary trees based on their structure and properties.

1. Full Binary Tree:

A binary tree in which every node has either **0 or 2 children**.

No node in a full binary tree has only one child.

Example:

```
1
/ \
2   3
/ \
4   5
```

Properties:

1. Total nodes = $2^h - 1$ (where h is the height of the tree).
2. All non-leaf nodes have exactly 2 children.

2. Complete Binary Tree:

A binary tree in which:

All levels, except possibly the last, are completely filled.

The last level has nodes filled from **left to right**.

Example:

```
      1
     / \
    2   3
   / \
  4   5
```

Properties:

1. All leaf nodes are as far left as possible.
2. Commonly used in heap implementations.

3. Perfect Binary Tree:

A binary tree in which all internal nodes have **2 children**, and all leaf nodes are at the **same level**.

Example:

```
      1
     / \
    2   3
   / \ / \
  4  5 6  7
```

Properties:

1. Total nodes = $2^h - 1$, where h is the height.
2. Number of leaf nodes = 2^{h-1}.

4. Skewed Binary Tree:

A binary tree where all nodes have only one child.

Can be:

Left-Skewed:

All children are on the left.

Right-Skewed:

All children are on the right.

Example (Left-Skewed):

```
      1
     /
    2
   /
  3
```

Properties:

1. Height of the tree = Number of nodes - 1.
2. Degrades to linear structure, similar to a linked list.

5. Balanced Binary Tree:

A binary tree in which the **height difference between the left and right subtrees of every node** is at most 1.

Example: AVL Tree

Properties:

1. Ensures logarithmic time complexity for search, insertion, and deletion.
2. Commonly used for efficient searching.

6. Binary Search Tree (BST):

A binary tree where:

The left subtree of a node contains only nodes with values **less than** the node's value.

The right subtree of a node contains only nodes with values **greater than** the node's value.

Example:

```
    10
    / \
   5   20
  / \
 2   8
```

Properties:

1. Provides efficient searching, insertion, and deletion $(O(\log n) in\ average\ cases)$.
2. In-order traversal yields a sorted sequence.

4.4 Traversing of binary tree:

Definition:

Tree traversal refers to the process of visiting all the nodes of a binary tree in a specific order. During traversal, each node is processed (e.g., its value is printed, accessed, or modified) exactly once.

Types of Tree Traversals:

Tree traversals are mainly categorized into **two types**:

1. Depth-First Traversal:

Depth-first traversal explores the depth of a binary tree by visiting nodes in one branch before moving to another. There are three subtypes:

a) Preorder Traversal (NLR):

Process the **Node**, then traverse the **Left subtree**, and finally traverse the **Right subtree**.

Algorithm:

1. Visit the root.
2. Traverse the left subtree in preorder.
3. Traverse the right subtree in preorder.

Example:

For the binary tree:

```
      1
     / \
    2   3
   / \
  4   5
```

Preorder: `1, 2, 4, 5, 3`

b) Inorder Traversal (LNR):

Traverse the **Left subtree**, process the **Node**, and then traverse the **Right subtree**.

Algorithm:

1. Traverse the left subtree in inorder.
2. Visit the root.
3. Traverse the right subtree in inorder.

Example:

For the binary tree:

```
      1
     / \
    2   3
   / \
  4   5
```

Inorder: `4, 2, 5, 1, 3`

c) Postorder Traversal (LRN):

Traverse the **Left subtree**, then the **Right subtree**, and finally process the **Node**.

Algorithm:

1. Traverse the left subtree in postorder.
2. Traverse the right subtree in postorder.
3. Visit the root.

Example:

For the binary tree:

```
      1
     / \
    2   3
   / \
```

4 5

Postorder: `4, 5, 2, 3, 1`

2. Breadth-First Traversal (Level Order Traversal):

In breadth-first traversal, nodes are visited level by level, starting from the root and moving to the next level.

Algorithm:

1. Start at the root node.
2. Visit all nodes at the current level.
3. Move to the next level and repeat.

Example:

For the binary tree:

```
      1
     / \
    2   3
   / \
  4   5
```

Level Order: `1, 2, 3, 4, 5`

Applications of Tree Traversals:

1. Preorder:

Used to create a copy of the tree or prefix expressions.

2. Inorder:

Used for binary search trees (BST) to retrieve data in sorted order.

3. Postorder:

Used for deleting a tree or evaluating postfix expressions.

4. Level Order:

Used in breadth-first search (BFS) algorithms.

4.5 Header Nodes, Threads:

1. Header Nodes:

Definition:

A **header node** is a special node used in certain data structures (like threaded binary trees or linked lists) to serve as a placeholder or marker. It often helps in simplifying operations like traversal, insertion, and deletion by providing a consistent entry point to the structure.

Usage in Binary Trees:

1. Acts as a starting point for traversal.

2. Contains pointers to the root of the tree and other metadata, such as tree size or height.
3. Can help manage special cases like empty trees.

Example:

In a threaded binary tree, the header node might point to the root of the tree and facilitate circular links.

2. Threads in Binary Trees:

Definition:

Threads are pointers added to a binary tree's nodes to make traversal faster and avoid the use of recursion or stacks. These pointers replace `NULL` links in leaf nodes and point to the **inorder predecessor** or **successor** of the node.

Threaded Binary Tree:

A **threaded binary tree** is a binary tree where threads are introduced to help traverse the tree in an efficient manner.

Types of Threaded Binary Trees:

1. Single Threaded:

A node contains a thread pointer to either its **inorder predecessor** or **inorder successor**.

2. Double Threaded:

A node contains two thread pointers: one for the **inorder predecessor** and one for the **inorder successor**.

Advantages of Threads:

1. Eliminates the need for recursion or a stack during traversal, saving memory.
2. Traversal becomes faster as threads directly connect nodes in the traversal order.

Example of Inorder Threading:

Consider this binary tree:

```
  1
 / \
2   3
```

Threaded Representation:

The left thread of `2` points to its **inorder predecessor** (if any).
The right thread of `3` points to its **inorder successor** (if any).

Applications of Threads:

1. Useful in tree traversal when memory usage needs to be minimized.
2. Efficient for in-order traversal of binary search trees.

4.6 General Tree Introduction:

Definition:

A **general tree** is a hierarchical data structure where each node can have an arbitrary number of children. Unlike binary trees, which limit nodes to two children, general trees allow flexibility in the number of descendants a node can have.

Key Characteristics of General Trees:

1. Node:

The basic element of the tree that holds data and references to its children.

2. Parent and Child Relationship:

Each node can have one parent and multiple children.

3. Root Node:

The topmost node with no parent.

4. Leaf Node:

A node with no children.

5. Internal Node:

A node with at least one child.

6. Height of the Tree:

The longest path from the root to a leaf.

7. Degree of a Node:

The number of children a node has.

Tree Representation in Memory:

General trees can be represented in memory using two common methods:

1. Parent-Child Representation:

Each node contains a pointer to its **first child** and a pointer to its **next sibling**.
This is known as the **first-child, next-sibling representation**.

Example:

For this general tree:

```
    A
   / | \
  B  C  D
```

Memory Representation:

A -> First Child: B
B -> Next Sibling: C
C -> Next Sibling: D
D -> No Sibling

2. Dynamic Lists:

Each node contains a dynamic list (or array) of pointers to its children.

Applications of General Trees:

1. **Hierarchical Data:**
   Organization charts, file systems, and XML/HTML documents.

2. **Game Trees:**
   Representing possible moves in a game like chess.

3. **Parsing Trees:**
   Used in compilers for parsing expressions or code.

Advantages of General Trees:

1. Flexibility in the number of child nodes.
2. Suitable for hierarchical and structured data.

Disadvantages of General Trees:

1. Traversal and manipulation can be more complex compared to binary trees.
2. Memory management requires dynamic structures for arbitrary child nodes.